

Computer Architectures

Exam of 03/02/2026

B3

Please read the following notes before proceeding

- 1) It is recommended to save the project on the C drive during development. Once you have completed your work, you must save the project inside the exam folder on the Z drive (the same folder where you found this document). **Projects saved elsewhere will not be collected and will be permanently lost.**
- 2) The assembly subroutine developed in the first exercise must comply with the ARM Architecture Procedure Call Standard (AAPCS) standard (in terms of parameter passing, returned value, callee-saved registers).
- 3) If you prefer, you can begin with the second exercise. Since it requires calling the assembly subroutine from the first exercise, you can create a stub in the assembly file to proceed. For example:

```
subroutineName    PROC
    ; to do: add code
    MOV r0, #10    ; example of a return value
    BX LR
    ENDP
```

- 4) To earn points on all exercises, it is recommended that you write at least some code for each.
- 5) **The final project must run on the actual board (e.g., button debouncing is required).**

Special characters with Italian keyboards

- Square brackets []
Left bracket: ATL GR + [
Right bracket: ALT GR +]
- Curly brackets { }
Left bracket: LSHIFT + ALT GR + [
Right bracket: LSHIFT + ALT GR +]
- Hash #: ALT GR + à

Bitwise operations in C

Left shift by N bits = $A \ll N$
Right shift by N bits = $A \gg N$

A XOR B = $A \wedge B$
A AND B = $A \& B$ (&& is the logical AND)
A OR B = $A | B$ (|| is the logical OR)

Question 1 (8 points)

The Recamán's sequence $a_0, a_1, a_2 \dots$ is defined as follows:

$$a_n = \begin{cases} 0 & \text{if } n = 0 \\ a_{n-1} - n & \text{if this value is positive and has not appeared earlier in the sequence} \\ a_{n-1} + n & \text{otherwise} \end{cases}$$

The first terms of the sequence are:

0, 1, 3, 6, 2, 7, 13, 20, 12, 21, 11, 22, 10, 23, 9, 24, 8, 25, 43, 62, 42, 63, 41, 18, 42, 17, 43, 16, 44, 15, 45, 14, 46, 79, 113, 78, 114, 77, 39, 78, 38, 79, 37, 80, 36, 81, 35, 82, 34, 83, 33, 84, 32, 85, 31, 86, 30, 87, 29, 88, 28, 89, 27, 90, 26, 91, 157, 224, 156, 225, 155, ...

Write the `Recaman` subroutine in ARM assembly language. You must follow this prototype:

void Recaman(uint32_t* area, uint8_t n);

The subroutine receives:

- the address of an uninitialized read-write data area
- the number of elements n .

The subroutine must fill the data area with the first n elements of the Recamán's sequence (starting with a_0). Each element is stored as a word: you have to reserve space in the data area properly.

Question 2 (10 points)

Add the following functionalities to your project.

1) Convert the analog value of the potentiometer to a digital value using the Analog-to-Digital Converter (ADC). The ADC produces a 12-bit digital output. Display the 8 most significant bits of this result on the LEDs. LED 4 corresponds to the most significant bit, and LED 11 corresponds to the least significant bit (of the displayed 8-bit value).

2) When the user presses the `Key2` button, call the `Recaman` subroutine passing:

- an uninitialized array of integers
- the 8 most significant bits produced by the last ADC conversion (i.e., the 8-bit value currently displayed on the LEDs)

The `Recaman` subroutine fills the array with the elements of the Recamán sequence. Immediately display the 8 least significant bits of the first element of the array on the LEDs (that is, since the first element is zero, all LEDs must be turned off). Then, display the 8 least significant bits of each subsequent element at 2-second intervals.

Note: the array may be defined statically, since its maximum number of elements is 255 (corresponding to the case in which the 8 most significant bits of the last ADC conversion are equal to 0xFF).